File Streaming and Unicode Tests

Raspberry Pi 3 – Team C

Guillem Moreno

October 2026

1 Scope

This directory provides equivalent Python 3.10+ and C++17 implementations for reading a file as bytes, supplying a transport stream, reconstructing the original file, identifying Unicode encoding candidates and producing a separate UTF-8 text copy. Both endpoints may run Raspberry Pi OS on a Pi 3.

The transmitted stream is exactly the source file, including any BOM, NUL bytes and original line endings. It contains no new header, filename or encoding tag. The caller supplies the original byte count and CRC-32 separately. These values are application metadata for this test harness, not a proposed radio frame format. CRC-32 detects accidental corruption; it is not authentication.

The implementation uses bounded buffers: 4,096 bytes by default, configurable from 1 byte to 1 MiB. Inspection makes several streaming validation passes rather than loading the whole file. Source files must remain unchanged during inspection and transfer. A size/CRC check detects changes between preparation and transmission.

2 Build and run on Raspberry Pi OS

Run from the repository root:

```
sudo apt-get install build-essential python3
make -C proto/test_encoding_text test
cd proto/test_encoding_text
python3 generate_samples.py build/samples
python3 encoding_stream.py inspect build/samples/utf-8_bom.txt
./build/encoding_stream inspect build/samples/utf-8_bom.txt
```

The Python implementation uses only the standard library. The C++ implementation has no external library dependencies. Build it with a native Pi compiler; the supplied CI also builds ARM32 hard-float and AArch64 executables. Cross-builds are compile checks, not evidence of execution on a Pi or a physical USB stick.

Both command-line programs expose the same commands and emit JSON results. Use `-help` for arguments. The examples below use the Python script. The C++ executable is `build/encoding_stream`.

```
python3 encoding_stream.py discover /media/pi/INPUT
python3 encoding_stream.py copy input.txt recovered.txt
python3 encoding_stream.py send input.txt stream.bin
```

Discovery accepts exactly one regular `.txt` file in the specified directory (case-insensitive extension). It ignores directories and symlinks, and rejects zero or multiple candidates. An explicit source path bypasses discovery.

`send` prints the byte count and CRC-32. Pass those exact values to `receive`; for example, the nine bytes `123456789` have length 9 and CRC-32 `cbf43926`:

```
python3 encoding_stream.py receive stream.bin recovered.txt \
    --size 9 --crc32 cbf43926
```

Use - as the stream path to read standard input or write standard output. When sending to stdout, metadata is printed to stderr so it cannot contaminate the byte stream. The stream may be moved between machines by the existing transport; a pipe or stream file is sufficient for local testing.

3 Encoding selection

The supported encodings are UTF-8, UTF-16 LE/BE and UTF-32 LE/BE. The operating system reads bytes; the application chooses how those bytes are interpreted. The inspection result includes all matching BOM signatures, all syntactically valid encodings, the automatic selection if available, and a detection status.

Status	Meaning
`bom`	One valid BOM interpretation.
`ambiguous_bom`	Multiple valid interpretations of an initial signature.
`invalid_bom`	Recognised signature, but no valid matching text.
`unique_valid`	Exactly one valid candidate within the five supported encodings.
`ambiguous`	No BOM and several valid candidates.
`invalid`	No supported encoding validates.
`empty`	No bytes from which to infer an encoding.

Automatic selection is conservative. Missing BOMs do not imply UTF-8. For example, `41 00` can be UTF-8 A followed by NUL or UTF-16 LE A. Similarly, `FF FE 00 00 41 00 00 00` permits a UTF-32 LE BOM plus A, or a UTF-16 LE BOM plus NUL, A, NUL. Strict automatic decoding rejects this overlap. Use an explicitly declared encoding to resolve it:

```
python3 encoding_stream.py decode recovered.txt display.txt \
    --encoding utf-16-le --chunk-size 3
```

The optional `-bom-policy longest` applies a conventional longest-signature rule. It is suitable only when that convention is agreed for the input files; it does not prove the source encoding. A unique valid candidate without a BOM is also a conclusion limited to the supported set, not a universal detector.

Decoding validates strictly: malformed sequences, truncated code units, isolated surrogates, overlong UTF-8 and values above U+10FFFF cause an error. NUL and Unicode noncharacters remain valid. The decoder maintains state across chunks, including split UTF-8 sequences and UTF-16 surrogate pairs.

The decoded output is a separate UTF-8 file. One initial BOM is consumed by default; `-keep-bom` retains the initial U+FEFF as text. Interior U+FEFF, combining sequences and line endings are preserved. The original file is never normalised, re-encoded or overwritten.

4 Receiver and filesystem behaviour

`FileReceiver` accepts ordered data with byte offsets. Gaps, duplicate offsets, out-of-order chunks and excess data are rejected. The transport must perform ordering, deduplication and retries before delivering bytes to this layer. Finalisation checks the expected length and CRC-32, flushes and syncs the file, then reads it back for verification.

Data is first written to `destination.part`, created exclusively. An existing final file or partial file is never overwritten. On success, Linux `renameat2(RENAME_NOREPLACE)` publishes the result, and the parent directory is synced. Raspberry Pi OS must provide this operation on the destination filesystem. Unsupported publication or sync operations are reported as errors.

A normal exception removes the partial file owned by that operation. A killed process or power loss can leave a partial file; inspect it and remove it explicitly before retrying. This test layer does not implement resume. If directory sync fails after publication, the complete final file can exist although the operation reports failure. It must be inspected before a retry. Safely unmount USB media after files are closed.

Incomplete transfers are not published as complete files. The existing competition node can save a received prefix at STOP; that policy is separate from this complete-file receiver and must not be silently replaced by it.

5 API integration

Python exposes `file_info`, `file_chunks`, `send_file`, `FileReceiver`, `inspect_file` and `decode_file`. C++ exposes equivalent functions in namespace `mtp_text`.

```
# Python: ordered payload delivery by the existing transport.
info = file_info(source_path)
with FileReceiver(destination_path, info) as receiver:
    for offset, payload in transport_payloads:
        receiver.accept(offset, payload)
    receiver.finish()
```

Here `transport_payloads` represents the transport adapter, not a function implemented by this test package. The C++ sender takes a byte-sink callback; the receiver takes `accept(offset, data, length)`. Both expose a finalisation call that the adapter invokes only when transfer completion has been established.

The repository already supplies `proto/src/codec.c` and `proto/port/linux/store.c`. When using its compression path, compression operates on the original byte stream and decompression precedes this receiver's file checks. Existing radio frames, retransmission state and compressed-stream metadata are unchanged. This package is a test harness and reusable API; it is also linked into `teamc-node` through the C interface in `encoding_stream_c.h`. The node's C sources compile as C; only the file/Unicode implementation and its bridge compile as C++17.

This harness uses files on a mounted filesystem. The existing node accesses its USB stick through FatFs as a raw block device. Do not use both paths concurrently on the same mounted device. Integration must use one ownership model consistently.

5.1 The integrated node

During SETUP, FatFs reads the USB file without interpreting its contents. The C++ receiver verifies staged SD-card caches `in.bin` and `tx.bin` before they replace the previous caches. At GO, the sender reads the selected cache through `send_file`, checking its stored size and CRC-32. The node retains its existing whole-file buffers and 4 MiB input limit; linking the streaming API does not make the full node use constant memory.

At STOP, a complete SRI/MRM stream must match the CRC-32 announced in HELLO. A compressed complete file must also reach the zlib stream end, pass its checksum, consume all input and fit the 4 MiB output limit. The recovered bytes are saved through `FileReceiver` to `received.bin` on the SD card, then written unchanged to the USB stick through FatFs with its existing read-back check. NM uses the same cache/text path once a full file is available.

The node's internal cache adapter stages `path.new` using the ordinary no-overwrite receiver, validates it and then atomically replaces `path`. This replacement applies to application-owned caches only. Existing staging files are refused and may need inspection after a crash. A directory-sync error can be reported after publication. The radio thread's resume store is unchanged.

Encoding inspection produces `in.encoding.json` at SETUP and `received.encoding.json` after complete reception. A successful strict decode produces a separate `in.utf8.txt` or `received.utf8.txt` on the SD card. Neither reports nor decoded copies are written onto the competition USB. Ambiguous or malformed text still transfers and saves in its original form. Decoding failures are logged; the original remains available.

```
make -C proto linux
proto/build/teamc-node --data /data/teamc \
    --text-encoding auto --text-bom-policy strict
# A format explicitly agreed for the input files:
proto/build/teamc-node --data /data/teamc \
    --text-encoding utf-16-le
```

The default is strict automatic selection. These optional settings disable text processing or select the previously described longest-signature convention:

```
--text-encoding none
```

```
--text-bom-policy longest
```

These options affect text views only. Discovery on the raw USB stick keeps the node's existing largest-`.txt` rule; the standalone directory-discovery command still requires a single candidate.

STOP before completion retains the existing prefix-scoring behaviour. A raw prefix, or the decompressed prefix of an incomplete compressed stream, is saved to USB and cached as `received-prefix.bin`. It is not automatically decoded because it may end inside a character. Any previous UTF-8 view and inspection report are removed. `received.bin` may still hold the last complete round; the prefix cache records this partial round. Malformed or truncated streams are never classified as complete merely because a decompressor produced some bytes.

6 Validation

`make test` builds the C++ program and API checks, then runs Python unit tests and cross-language tests. Cases include all five encodings with and without BOMs; empty and BOM-only files; NUL, combining marks, emoji and mixed line endings; overlapping and incomplete signatures; malformed sequences; short writes; truncation, corruption and excess data; existing destinations and publication races; offsets and file discovery. Differential tests compare both inspectors on fixed and deterministic random inputs. Streaming tests cover multi-megabyte files.

The node integration suite runs with:

```
make -C proto encoding-e2e
```

It checks the C ABI and complete-stream zlib validation, then runs real sender/receiver processes over the UDP radio backend with FatFs stick images. It verifies all five BOM encodings, overlapping signatures with strict/declared/longest policies, invalid and BOM-less text, empty files, disabled decoding, sender-cache corruption and STOP during a partial transfer. `make -C proto e2e` additionally exercises the existing SRI, MRM, reboot and three-node NM scenarios. CI runs both suites.

Run the tests on both Pis and on the actual destination filesystem before hardware integration. Host tests and ARM compilation do not verify radio timing, physical USB behaviour or resilience to a real power interruption.